

Automated Tuberculosis Detection Using Convolutional Neural Networks (CNN) with Transfer Learning

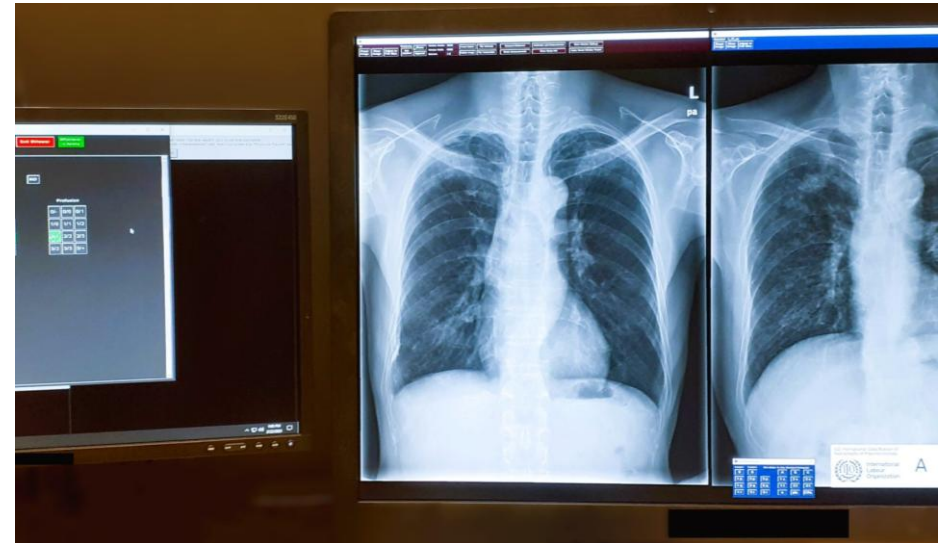
Presented by: Mary Nnipaa Meteku

Instructor: Dr. Neerav Kaushal

Problem Statement

- TB is still a leading infectious cause of death worldwide.
- Manual chest X-ray interpretation requires radiologists and is prone to:
 - ✓ Fatigue
 - ✓ Subjective judgement
 - ✓ Uneven expertise
- Low-resource regions lack radiology specialists.

•**Objective:** Use AI to assist early TB detection, not replace clinicians.



Dataset Used

- **Source:** Kaggle

-

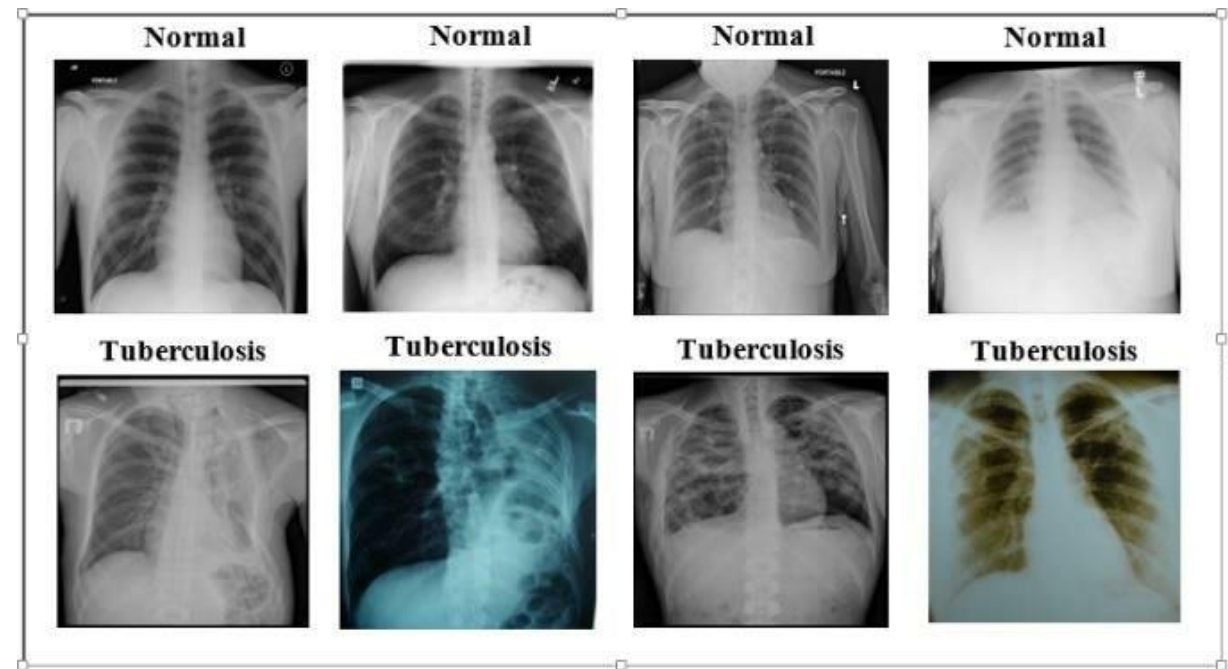
Dataset: Tuberculosis Chest X-Rays Images (Yasser Hessein)

-

Link:

<https://www.kaggle.com/datasets/yasserhessein/tuberculosis-chest-x-rays-images>

-



Class	Images
TB Positive	2,494
Normal	514
Total	3,008

Exploratory Data Analysis

- **Class Imbalance Analysis**

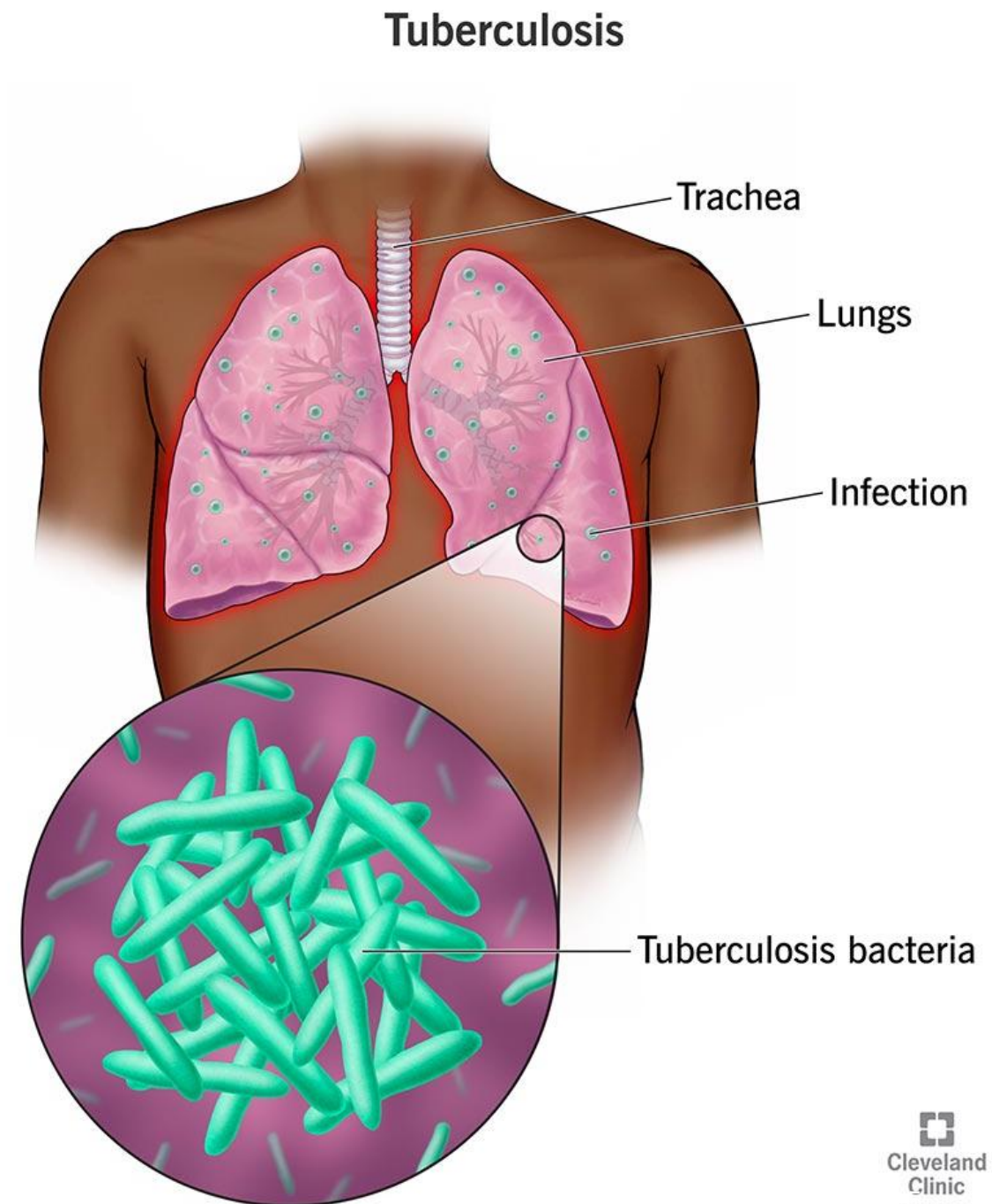
The 5:1 ratio mirrors real clinical conditions where TB prevalence is significantly higher than normal cases in screening populations.

- **Clinical Implications**

Dataset imbalance requires specialized evaluation metrics prioritizing recall and F1-score over simple accuracy measurements.

Why This Matters

- Missing TB cases = life-threatening consequences
- False positives are manageable → re-evaluation.
- False negatives = continued infection & delayed care.
- Priority metric: Recall (Sensitivity).



Data Splitting

- Training Set: 70%

Primary dataset for model learning and parameter optimization with stratified sampling.

- Validation Set: 15%

Model selection and hyperparameter tuning with early stopping.

- Test Set: 15%

Final model evaluation and performance assessment on unseen data.

Preprocessing

Image Resizing

All images resized to 224x224 pixels to match EfficientNet input requirements and ensure consistency.

Label Encoding

Binary encoding applied with 0 for Normal cases and 1 for TB-positive cases for classification.

Data Augmentation

Horizontal flipping, rotation ($\pm 10-15^\circ$), zoom ($\pm 10\%$), width and height shift (≤ 0.1) to simulate clinical variations.

Stratified Splitting

3-way stratified split maintaining class distribution across training, validation, and test sets for balanced evaluation.

tb



Random TB & Normal X-rays

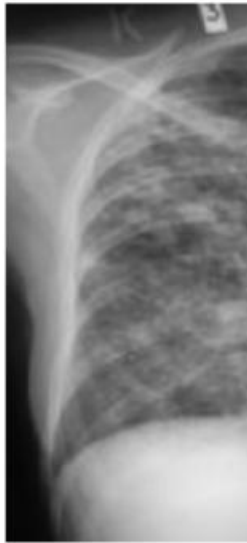
tb



tb



tb



normal

normal



Transfer Learning Model: EfficientNetB0

Why EfficientNetB0

- High accuracy with fewer parameters
- Lightweight → runs in VM
- Strong generalization

Architecture

- EfficientNetB0 (Frozen)
- GAP (GlobalAveragePooling2D)
- Dense(128, ReLU)
- Dropout(0.45)
- Dense(1, Sigmoid)

Hyperparameters

Parameter	Value	Justification
Optimizer	Adam	Adaptive learning rate
Batch Size	32	Memory constraint balance
Learning Rate	1e-4	Transfer learning stability
Epochs	10-15	Early stopping prevention
Dropout Rate	0.45	Overfitting prevention
Loss Function	Binary Cross-entropy	Binary classification task

Hyperparameter Tuning

- **Tuning Methods**
- Learning Rate adjustment
- Different batch sizes
- Gradual unfreezing of EfficientNet layers
- Controlled augmentation intensity

Observed

- Lower LR → smoother convergence
- Unfreezing too early → overfitting
- Dropout 0.45 → optimal balance

```
# Train the model using the training dataset and validate on the validation
EPOCHS = 10

history = model.fit(
    train_gen,
    epochs=EPOCHS,
    validation_data=val_gen,
    callbacks=callbacks
)

packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning:
()

/step - accuracy: 0.8043 - loss: 0.5275
inf to 0.47638, saving model to tb_efficientnet_best.h5
model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
3s/step - accuracy: 0.8047 - loss: 0.5271 - val_accuracy: 0.8293 - val_loss: 0.47

/step - accuracy: 0.8365 - loss: 0.4765
e from 0.47638
3s/step - accuracy: 0.8363 - loss: 0.4766 - val_accuracy: 0.8293 - val_loss: 0.48

/step - accuracy: 0.8196 - loss: 0.4996
0.47638 to 0.47024, saving model to tb_efficientnet_best.h5
model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
#####
3s/step - accuracy: 0.8197 - loss: 0.4994 - val_accuracy: 0.8293 - val_loss: 0.47

# Unfreeze some of the top layers of the base model
base_model.trainable = True

fine_tune_at = int(len(base_model.layers) * 0.7) # freeze bottom ~70%

for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

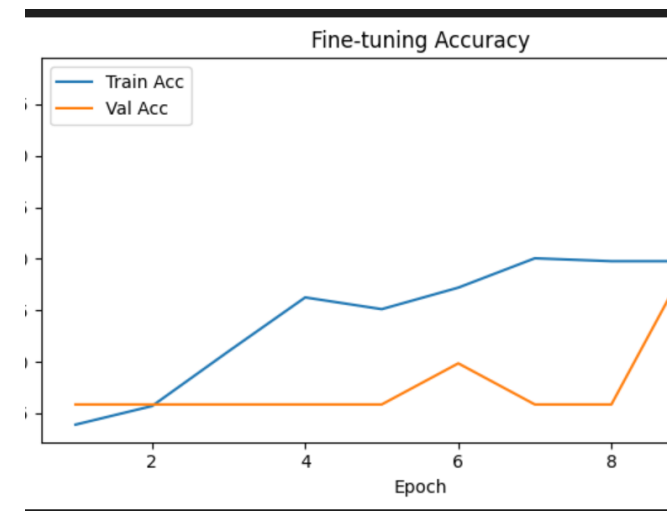
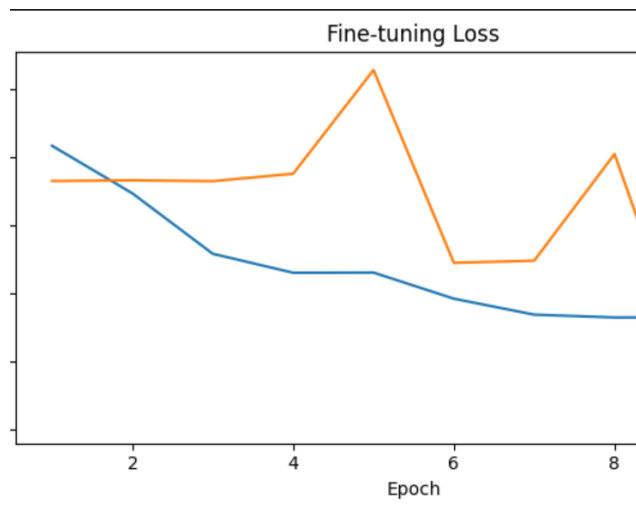
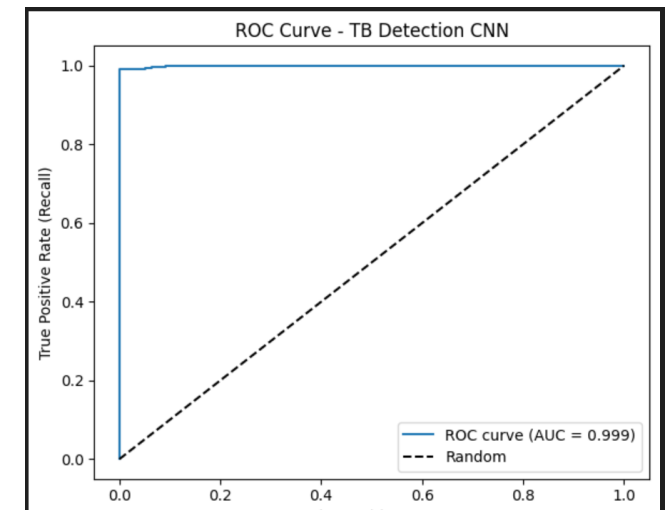
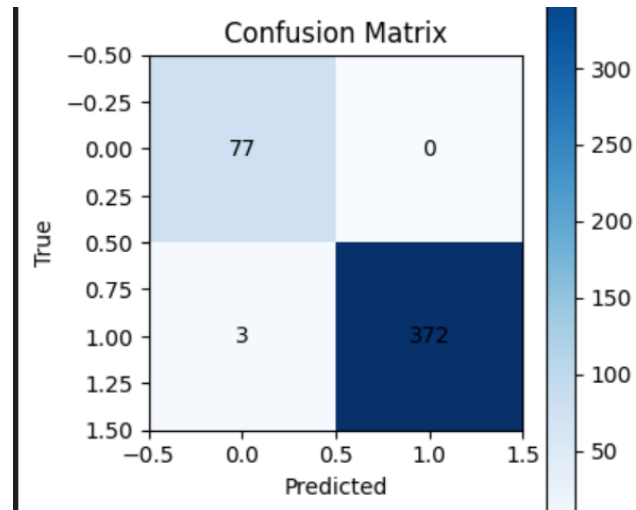
history_ft = model.fit(
    train_gen,
    epochs=10, # extra epochs for fine-tuning
    validation_data=val_gen,
    callbacks=callbacks
)

Epoch 1/10
66/66 ----- 0s 4s/step - accuracy: 0.8223 - loss: 0.5227
Epoch 1: val_loss improved from 0.46469 to 0.46461, saving model to tb_efficientnet_best.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving
66/66 ----- 310s 4s/step - accuracy: 0.8223 - loss: 0.5226 - val_accuracy:
Epoch 2/10
66/66 ----- 0s 4s/step - accuracy: 0.8260 - loss: 0.4672
Epoch 2: val_loss did not improve from 0.46461
66/66 ----- 274s 4s/step - accuracy: 0.8261 - loss: 0.4669 - val accuracy:
```

Model Performance - EfficientNet Results

Metric	CNN	EfficientNet
Accuracy	0.84	0.99
Precision	0.85	1.00
Recall	0.88	0.99
F1-score	0.86	0.99
ROC-AUC	0.92	0.99

Cont... Model Performance - EfficientNet Results



Reflections and Future Work

- AI **assists**, not replaces clinicians
- Medical ML requires:
 - Trust
 - Explainability
 - Ethical safety
- Missing a TB case = unacceptable
 - ❖ Multi-class: TB + Pneumonia + COVID
 - ❖ Explainable AI: Grad-CAM heatmaps
 - ❖ Deployment on mobile devices for Low- and Middle-Income Countries.



THANK YOU